The Journal of Open Source Software

ChronosLang: A Python-inspired compact, experimental programming language with native time-travel debugging, temporal variables, deterministic concurrency, compile-time macros, runtime reflection, static typing, built-in testing, and lightweight probabilistic/ML modules

Srijon Kumar Shill [1]

1 Independent Researcher, Bangladesh

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: George K. Thiruvathukal

Submitted: 08 December 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

ChronosLang is a compact, Python-inspired experimental programming language (Van Rossum & Drake, 2009) and interpreter that makes **time, concurrency, and introspection** first-class concepts. It is built around three interlocking capabilities:

1. **Time-native programming.** Variables may be declared as *temporal* and carry a time-indexed history; assignments can be scheduled into the future (e.g., `x = 5 @ t+2s`), and a global `Timeline` object allows programs and tools to move forward and backward in logical time. Temporal variables integrate tightly with a command-line time-travel REPL and a PyQt5 debugging GUI (Riverbank Computing, 2023).

2. **Deterministic concurrency.** ChronosLang provides lightweight concurrency inspired by communicating sequential processes (Hoare, 1978) and Go-style channel-based communication (Pike, 2012). Goroutines (`go expr`) and unbuffered rendezvous channels (`make(chan T)`, `<-ch`, `ch <- v`) allow deterministic concurrent examples suitable for teaching and research.

3. **Compile-time macros and runtime reflection.** Hygienic-ish macros allow AST-level compile-time transformations, and a reflection API enables programs to inspect globals, locals, types, and temporal state. The design follows classic metaobject protocol ideas (Kiczales et al., 1991) while remaining small and inspectable.

ChronosLang further includes:

- a conservative **static type checker** inspired by foundational type theory (Milner, 1978),
- a built-in **unit test framework** aligned with test-driven development principles (Beck, 2003),
- a lightweight **probabilistic module** inspired by Church-style generative models (Goodman et al., 2008),
- a **tensor/ML module** for small machine learning experiments related to modern deep learning concepts (LeCun et al., 2015),
- and two complementary tools: a **PyQt5 time-travel debugger GUI** and a **FastAPI-based web runner**.

ChronosLang is open source (MIT license), intentionally compact, and designed for use as a **teaching language** and **research testbed** for temporal semantics, reversible debugging,

concurrency, and language-integrated probabilistic/ML features.

Statement of need

Students and researchers in programming languages, debugging, distributed systems, and AI often need **small, self-contained interpreters** that are easy to read, modify, and extend, so they can teach core ideas or prototype new semantics without the overhead of a large compiler/runtime. Existing ecosystems provide powerful components, but the specific combination needed for *temporal semantics + introspection + minimal concurrency + lightweight probabilistic/ML demos* is usually scattered across different tools:

- General-purpose languages (e.g., Python) and libraries (e.g., NumPy/PyTorch) support scientific computing and ML, but they do not provide a **language-level temporal state model** with scheduled assignments and timeline scrubbing as part of the language's core syntax/semantics (LeCun et al., 2015; Van Rossum & Drake, 2009).
- Time-travel debugging and record/replay tools exist, but they are typically tied to a particular runtime and are harder to repurpose as a **teaching interpreter** where time-indexed state is an explicit, programmable concept (Berger et al., 2006; Zeller, 2002).
- Probabilistic programming systems (e.g., Church) demonstrate compact modeling and inference ideas, but they do not aim to be a single, small interpreter that also includes temporal state, channels/goroutines, macros, and reflection for teaching and prototyping (Goodman et al., 2008).

ChronosLang addresses this gap by providing a compact language and Python interpreter built to support **teaching, demos, and small-scale experiments** in one place.

What ChronosLang provides (as implemented)

- **Temporal variables and scheduled updates.** Programs can declare temporal state (`temporal x = ...`) and schedule future assignments using `@ t+Ns` (e.g., `x = 5 @ t+2s`). Temporal variables store a history of `(time, value)` pairs, and the runtime maintains an explicit logical `current_time`. Advancing logical time applies scheduled events in timestamp order; moving backward changes the time pointer and reads older values from stored history.
- **Time-travel inspection (CLI + GUI).** ChronosLang includes a CLI debugger (`--time-travel`) with commands such as `forward`, `back`, `show`, and `history`, and a PyQt5 GUI prototype that runs a program's prelude once and then supports slider-based timeline scrubbing and variable history viewing.
- **Minimal concurrency for demonstrations.** The language supports `go` for background execution and unbuffered rendezvous channels (`make(chan T)`, `ch <- v`, `<-ch`) inspired by CSP/Go-style communication (Hoare, 1978; Pike, 2012). The implementation is intentionally small (Python threads + a rendezvous `Channel`) so the mechanism is readable and suitable for classroom examples and short demos.
- **Compile-time macros + runtime reflection.** ChronosLang supports compile-time macros (`macro name(...)`) implemented by collecting macro definitions and expanding macro calls before runtime. At runtime, a reflection object (`reflect`) plus helper functions (`reflect_type`, `reflect_globals`, `reflect_func_name`, `reflect_locals`) allow programs to inspect global names, function signatures, the current timeline state, and locals during function execution (Kiczales et al., 1991).
- **Built-in probabilistic and ML mini-modules.** The interpreter includes a lightweight `prob` module (uniform/normal/bernoulli/binomial + inference via importance sampling or Metropolis–Hastings) and an `ml` module providing a minimal tensor API and linear regression training. The ML implementation optionally uses PyTorch autodiff when available, otherwise it falls back to NumPy closed-form least squares (Goodman et al., 2008; LeCun et al., 2015).

How this helps educators (concrete classroom value)

ChronosLang is useful for educators because it is **small enough to fit into a course module** while still demonstrating multiple "real" language/runtime ideas end-to-end:

- **Interpreter structure in one place:** parsing (Lark), a preprocessing step (indent → braces), AST evaluation, and a simple runtime environment. Students can understand and modify the interpreter without needing a full compiler toolchain.
- **Temporal semantics as a teachable concept:** instructors can show how "time-indexed state" differs from ordinary variables by using `temporal` + `@ t+Ns`, then letting students inspect histories with `history x` or GUI scrubbing. This makes state evolution visible and reduces reliance on ad-hoc logging.
- **Debugging concepts with an explicit model:** the CLI and GUI demonstrate how a timeline pointer and stored histories can support time-based inspection. This is a practical teaching artifact for debugging lectures and lab exercises.
- **Concurrency basics without large frameworks:** `go` and rendezvous channels let instructors demonstrate message passing, synchronization, and deadlocks with tiny examples (producer/consumer). Because the primitives are minimal, examples stay short and readable.
- **Meta-programming and introspection:** macros + reflection allow instructors to demonstrate compile-time transformation versus runtime inspection using one codebase and clear examples.

How this helps researchers (concrete research/prototyping value)

ChronosLang is useful for researchers because it provides a **modifiable baseline runtime** where new ideas can be implemented quickly:

- **Prototype temporal semantics and tooling:** researchers can extend the timeline model (event scheduling, different conflict rules, alternative history representations) and immediately test changes via the existing CLI and GUI timeline inspection.
- **Experiment with language instrumentation:** compile-time macros enable AST-level rewriting for lightweight instrumentation or program transformations, while runtime reflection supports debugging/inspection utilities without external tooling.
- **Rapid "concept demo" environment:** the built-in examples and minimal runtime make it suitable for producing small, reproducible demonstrations of semantics (temporal state, message passing, macro expansion, reflection outputs) in papers, talks, or lab prototypes.
- **Lightweight probabilistic/ML demos in the same interpreter:** the `prob` and `ml` modules enable small examples where inference/training results can be stored, printed, and combined with other language features without embedding into a larger host framework.

ChronosLang intentionally makes its limits explicit: "time travel" applies to **temporal variable histories driven by scheduled timeline events**, not to general reversal of all side effects or full replay of concurrent thread behavior. This keeps the implementation compact and readable while still enabling clear demonstrations of temporal semantics, inspection, and lightweight meta-programming.

System overview

ChronosLang's implementation is organized as a **five-phase pipeline** and a set of compact runtime abstractions.

Front end: syntax, preprocessor, and parser

ChronosLang uses indentation-based syntax similar to Python (Van Rossum & Drake, 2009). A minimal **indentation-to-braces preprocessor** inserts `{}` markers based on indentation levels,

DRAFT

simplifying grammar design. The resulting token stream is parsed by a **Lark** LALR parser into an AST supporting:

- function definitions and calls,
- temporal variables and scheduled assignments (`@ t+NUMBER s`),
- goroutines (`go expr`) and channel operations (`<-ch`, `ch <- v`),
- macros, test blocks, lists, and type expressions.

This front end is intentionally compact and easily extended.

Static type checking

The `TypeChecker` supports base types (`int`, `float`, `string`, `auto`), list types, and channel types. It performs return-type inference and verifies signature consistency based on classical type theory concepts (Milner, 1978). ChronosLang supports both:

- **strict mode**, where type errors abort execution,
- **permissive mode**, allowing coercions useful for notebooks or demos.

Runtime environment and temporal semantics

ChronosLang provides:

- lexical environments with parent scopes,
- first-class `Function` objects,
- a `Timeline` that stores scheduled events,
- `TemporalVar` objects that maintain timestamped histories.

Advancing time applies scheduled events, and temporal reads return values from the compact event history. Time may be moved backward without undoing state—values are accessed by querying history at earlier times.

This allows **reversible debugging** without requiring reverse execution, inspired by prior work on replay and debugging systems (Berger et al., 2006; Zeller, 2002).

Concurrency model

ChronosLang implements Go-style concurrency with CSP roots (Hoare, 1978):

- `go expr` spawns a thread,
- channels implement synchronous rendezvous (`send` waits for `recv`),
- interactions are deterministic and reproducible.

This makes concurrent examples easy to teach and reason about.

Macros and compile-time behavior

Compile-time macros allow AST-level transformations before execution, following the philosophy of metaobject protocols (Kiczales et al., 1991) but in a simpler form. Macros are expanded before runtime and do not appear in temporal histories or the time-travel debugger.

Reflection

The global `reflect` object provides:

- inspection of globals, locals, and types,
- inspection of temporal histories,
- access to function metadata,
- a list of compile-time macros,
- a helper `reflect_func_name()` for introspection.

This makes ChronosLang an excellent environment for studying language reflection and program structure.

Probabilistic and ML modules

`prob` module

Offers:

- uniform, normal, and Bernoulli distributions,
- binomial likelihood terms,
- inference via importance sampling or Metropolis–Hastings (Goodman et al., 2008).

`ml` and `tensor` modules

Support:

- tensor construction,
- matrix operations,
- MSE loss,
- linear regression training using PyTorch or NumPy,
- connections to modern ML techniques (LeCun et al., 2015).

Tooling: CLI, web runner, GUI debugger

ChronosLang includes:

- a **CLI runner**,
- a **FastAPI web runner** inspired by lightweight frameworks (Grinberg, 2018),
- a **PyQt5 GUI time-travel debugger** (Riverbank Computing, 2023) for inspecting temporal histories interactively.

Illustrative examples

The repository includes examples demonstrating:

DRAFT

- functions and arithmetic,
- typed and inferred functions,
- concurrency with channels,
- test blocks and assertions,
- temporal variables and scheduled updates,
- probabilistic inference with binomial data,
- ML linear regression,
- compile-time macros and runtime reflection.

These examples support both teaching and research use.

Impact and reuse

ChronosLang is not intended as a production language but as a **pedagogical and research platform** offering a compact, extensible environment for temporal semantics, debugging, concurrency, and probabilistic/ML features.

Its small codebase, extensive documentation (`spec.md`), and built-in tools make it well suited for:

- university courses,
- research prototypes,
- debugging/visualization studies,
- experimentation with temporal or reflective semantics.

Availability

ChronosLang is free and open source under the MIT License:

- Repository: https://github.com/Srijon25/ChronosLang
- Documentation: `README.md`, `spec.md`
- Examples: `examples/`
- Recordings & screenshots: `docs/`

Acknowledgements

ChronosLang builds on the Python ecosystem, including Lark, NumPy, PyTorch, FastAPI, Uvicorn, and PyQt5.

DRAFT

References

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley.

Berger, E. D., Zorn, B. G., & McKinley, K. S. (2006). Reconsidering the memory model for deterministic replay. *ACM SIGPLAN Notices*, *41*(6), 22–33.

Goodman, N. D., Mansinghka, V. K., Roy, D., Bonawitz, K., & Tenenbaum, J. B. (2008). Church: A language for generative models. *Proceedings of the 24th Conference on Uncertainty in Artificial Intelligence (UAI)*, 220–229.

Grinberg, M. (2018). *Flask web development* (2nd ed.). O'Reilly Media.

Hoare, C. A. R. (1978). Communicating sequential processes. *Communications of the ACM*, *21*(8), 666–677. https://doi.org/10.1145/359576.359579

Kiczales, G., Rivières, J. des, & Bobrow, D. G. (1991). *The art of the metaobject protocol*. MIT Press.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*, 436–444. https://doi.org/10.1038/nature14539

Milner, R. (1978). A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, *17*(3), 348–375. https://doi.org/10.1016/0022-0000(78)90014-4

Pike, R. (2012). *Concurrency is not parallelism*. https://talks.golang.org/2012/concurrency.slide.

Riverbank Computing. (2023). *PyQt5 reference guide*. https://www.riverbankcomputing.com/software/pyqt/intro.

Van Rossum, G., & Drake, F. L. (2009). *The python language reference manual* (3rd ed.). Python Software Foundation. https://www.python.org/doc/

Zeller, A. (2002). *Why programs fail: A guide to systematic debugging*. Morgan Kaufmann.

DRAFT